Name: Cohort/Batch: Date: Score:

DOMAIN-DRIVEN DESIGN PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Architecture

TOTAL: 10 QUESTIONS

Q1 What is Domain-Driven Design and what problem in Architecture does it solve? — Threat Model

Q2 What are the core components or concepts in Domain-Driven Design? — Architecture

Q3 How does Domain-Driven Design compare to its main alternatives? — Configuration

Q4 How do you get started with Domain-Driven Design for a new project? — Hardening

Q5 What configuration options most affect Domain-Driven Design's behavior in production? — SSO / IdP

Q6 How does Domain-Driven Design handle reliability and high availability? — Observability

Q7 What observability does Domain-Driven Design provide out of the box? — Lifecycle

Q8 What are common performance bottlenecks in Domain-Driven Design? — Compliance

Q9 What security best practices apply when running Domain-Driven Design? — Testing

Q10 What mistakes do teams commonly make when adopting Domain-Driven Design? — Tuning

ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Domain-Driven Design is a tool or

Mitigation

Domain-Driven Design is a tool or platform that automates or simplifies a specific concern in the Architecture domain, reducing manual effort and operational complexity for engineering teams.

A2 Domain-Driven Design has key building blocks

Primitives

Domain-Driven Design has key building blocks that work together: a configuration layer defines intent, a runtime layer enforces it, and an API or CLI layer allows operators to manage and observe the system.

A3 Domain-Driven Design trades off simplicity

Hardening

Domain-Driven Design trades off simplicity against feature richness differently than its alternatives. Choose Domain-Driven Design when its specific strengths performance, ecosystem, or ease of use align with your team's primary constraints.

A4 Install Domain-Driven Design via its package

Gotchas

Install Domain-Driven Design via its package manager or binary release. Follow the quickstart to initialize a project, configure the minimal required settings, and run the default command to verify the setup works end-to-end.

A5 Production-critical settings typically include

Federation

Production-critical settings typically include concurrency limits, timeout values, retry policies, resource limits, and logging levels. Review the official production hardening guide before deploying.

A6 Domain-Driven Design provides HA through

Observability

Domain-Driven Design provides HA through leader election, replication, or stateless horizontal scaling. Deploy at least two instances across availability zones and configure health checks for automatic failover.

A7 Domain-Driven Design exposes metrics

Automation

Domain-Driven Design exposes metrics (Prometheus endpoint or built-in dashboard), structured logs, and optionally distributed traces. Define SLOs on the key metrics and alert before users are affected.

A8 Performance bottlenecks typically stem from

Governance

Performance bottlenecks typically stem from misconfigured connection pools, large payload sizes, insufficient worker threads, or missing caches. Profile with built-in diagnostics before optimizing.

A9 Run Domain-Driven Design with the principle

Assessment

Run Domain-Driven Design with the principle of least privilege, enable TLS for all communication, use a secrets manager for credentials, and keep the software patched to the latest stable release.

A10 Common pitfalls include skipping the

Optimization

Common pitfalls include skipping the documentation and learning the API by trial and error, using default configurations in production, lacking a runbook for operational issues, and not testing failover scenarios.